```
import unittest
import sys
import os

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from shop import (
    get_low_stock, search_advanced, total_sum,
    add_to_cart, remove_from_cart, update_quantity, cart_total,
)


class TestGetLowStock(unittest.TestCase):
    def test_empty_list(self):
        self.assertEqual(get_low_stock([]), [])

    def test_no_low_stock(self):
        self.assertEqual(get_low_stock([{"name": "A", "sizes": {40: 10}}]), [])

    def test_one_low_stock(self):
        products = [
            {"name": "A", "sizes": {40: 1}},
            {"name": "B", "sizes": {40: 10}},
        ]
        result = get_low_stock(products)
        self.assertEqual(len(result), 1)
        self.assertEqual(result[0]["name"], "A")


class TestSearchAdvanced(unittest.TestCase):
    def setUp(self):
        self.products = [
            {"id": 1, "name": "Кеды футбольные", "category": "Игровые", "price": 2500},
            {"id": 2, "name": "Тапочки Crocs", "category": "Сланцы", "price": 800},
            {"id": 3, "name": "Кеды баскетбольные", "category": "Игровые", "price": 2500},
            {"id": 4, "name": "Сланцы", "category": "Пляжные", "price": 300},
        ]

    def test_search_by_name(self):
        self.assertEqual(len(search_advanced(self.products, "Кед")), 2)

    def test_search_by_category(self):
        self.assertEqual(len(search_advanced(self.products, "", category="Игровые")), 2)

    def test_search_by_price_range(self):
        result = search_advanced(self.products, "", min_price=600, max_price=2000)
        self.assertEqual(len(result), 2)


class TestTotalSum(unittest.TestCase):
    def test_empty_cart(self):
        self.assertEqual(total_sum([]), 0)

    def test_cart_with_products(self):
        products = [
            {"name": "A", "price": 100, "quantity": 2},
            {"name": "B", "price": 50, "quantity": 3},
        ]
        self.assertEqual(total_sum(products), 350)


class TestAddToCart(unittest.TestCase):
    def setUp(self):
        self.products = [{"id": 1, "name": "A", "sizes": {40: 5}, "price": 1000}]
        self.cart = []

    def test_invalid_size(self):
        result, message = add_to_cart(self.cart, self.products, 1, 99, 1)
        self.assertFalse(result)
        self.assertIn("размер", message.lower())

    def test_product_not_found(self):
        result, _ = add_to_cart(self.cart, self.products, 999, 40, 1)
        self.assertFalse(result)

    def test_division_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            _ = 1 / 0


class TestCart(unittest.TestCase):
    def setUp(self):
        self.products = [
            {"id": 1, "name": "Кед", "sizes": {40: 5, 41: 2}, "price": 3000},
            {"id": 2, "name": "Тапочки", "sizes": {0: 10}, "price": 800},
        ]
        self.cart = []

    def test_add_to_cart_success(self):
        ok, _ = add_to_cart(self.cart, self.products, 1, 40, 2)
        self.assertTrue(ok)
        self.assertEqual(self.cart[0]["quantity"], 2)

    def test_add_to_cart_increases_quantity(self):
        add_to_cart(self.cart, self.products, 1, 40, 1)
        add_to_cart(self.cart, self.products, 1, 40, 2)
        self.assertEqual(len(self.cart), 1)
        self.assertEqual(self.cart[0]["quantity"], 3)

    def test_remove_from_cart(self):
        add_to_cart(self.cart, self.products, 1, 40, 1)
        self.assertTrue(remove_from_cart(self.cart, 1, 40))
        self.assertEqual(len(self.cart), 0)

    def test_update_quantity(self):
        add_to_cart(self.cart, self.products, 1, 40, 1)
        self.assertTrue(update_quantity(self.cart, 1, 40, 5))
        self.assertEqual(self.cart[0]["quantity"], 5)

    def test_cart_total(self):
        add_to_cart(self.cart, self.products, 1, 40, 2)
        add_to_cart(self.cart, self.products, 2, 0, 3)
        self.assertEqual(cart_total(self.cart), 4400)


if __name__ == "__main__":
    unittest.main()
```